



Institut Universitaire des sciences

- Faculté des sciences et de technologie

✓ TD N°1 Modélisation

Nom & Prénom : COFFY Cliford

Niveau : L3

Prof. : Ismael SAINT AMOUR

Date : 28/03/2026

TD 1 : Génération de clés aléatoires

Objectif : Comprendre les ensembles de caractères et l'entropie.

Instructions :

1. Générer des clés de 8, 16 et 32 caractères avec les ensembles :

- `ascii_lowercase`
- `ascii_letters`
- `digits`
- `ascii_letters + digits + punctuation`

2. Calculer l'entropie de chaque clé. 3. Comparer les entropies et discuter quel type de clé est le plus sécurisé.

```
File Edit Selection View Go ... Search
modelisation.py 3, U X
C: > Users > User > Desktop > TD- > programmationweb > modelisation.py > ...
1  import string
2  import secrets
3  import math
4
5  # Fonction pour générer une clé aléatoire
6  def generate_key(length, charset):
7      return ''.join(secrets.choice(charset) for _ in range(length))
8
9  # Fonction pour calculer l'entropie
10 def calculate_entropy(key, charset_size):
11     return len(key) * math.log2(charset_size)
12
13 # Ensembles de caractères
14 charsets = {
15     "ascii_lowercase": string.ascii_lowercase,
16     "ascii_letters": string.ascii_letters,
17     "digits": string.digits,
18     "full": string.ascii_letters + string.digits + string.punctuation
19 }
20
21 # Longueurs de clés à tester
22 lengths = [8, 16, 32]
23
24 # Génération et affichage
25 for name, charset in charsets.items():
26     print(f"\nCharset: {name}")
27     for length in lengths:
28         key = generate_key(length, charset)
29         entropy = calculate_entropy(key, len(charset))
30         print(f"Length: {length}, Entropy: {entropy}")
```

● PS C:\Users\User> & C:/Users/User/AppData/Local/Programs/Python/Python314/python.exe c:/Users/User/

Charset: ascii_lowercase
Clé (8 caractères): umxhxdkd
Entropie: 37.60 bits
Clé (16 caractères): euzebdoeploiknhv
Entropie: 75.21 bits
Clé (32 caractères): ezmdnwoxqaxzqfoyzqflqcoofpyqpigwo
Entropie: 150.41 bits

Charset: ascii_letters
Clé (8 caractères): dVnjssBM
Entropie: 45.60 bits
Clé (16 caractères): EXJajCDwUzTSqMtY
Entropie: 91.21 bits
Clé (32 caractères): ROWoVyGVJEyfiuNXNouEgxjxnvelOBYm
Entropie: 182.41 bits

Charset: digits
Clé (8 caractères): 66017169
Entropie: 26.58 bits
Clé (16 caractères): 9704717868131012
Entropie: 53.15 bits
Clé (32 caractères): 79313551199323892692318001037530
Entropie: 106.30 bits

Charset: full
Clé (8 caractères): O#DU.+_"
Entropie: 52.44 bits
Clé (16 caractères): +JyHz1)l,tMF\EJ,
Entropie: 104.87 bits
Clé (32 caractères): l`;>=D`7z'2d4E2qdViA}FeS,WpWL+o{
Entropie: 209.75 bits

```
PROBLEMS 7 OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\User> & C:/Users/User/AppData/Local/Programs/Python/Python314/python.exe c:/Users/User/Desktop/TD-/programmationweb/n
Charset: ascii_lowercase
Clé (8 caractères): oavawjdi
Entropie: 37.60 bits
Clé (16 caractères): nufzutjqdgkqejcb
Entropie: 75.21 bits
Clé (32 caractères): jslpqxjurxjvxdavdwgllmtlvmdxymwg
Entropie: 150.41 bits

Charset: ascii_letters
Clé (8 caractères): qkDzyOak
Entropie: 45.60 bits
Clé (16 caractères): zfvKakyBboQNCQpq
Entropie: 91.21 bits
Clé (32 caractères): UkJLHlLvVnychSmUctXDUszaogUzTeDL
Entropie: 182.41 bits

Charset: digits
Clé (8 caractères): 45819114
Entropie: 26.58 bits
Clé (16 caractères): 1149275210391028
Entropie: 53.15 bits
Clé (32 caractères): 28253981508765813525805498646553
Entropie: 106.30 bits

Charset: full
Clé (8 caractères): wgeq7VD|
Entropie: 52.44 bits
Clé (16 caractères): k4`ndDuJX_t1?pDk
Entropie: 104.87 bits
Clé (32 caractères): }*vMn2Hu{grg(@`cLp5M?Id'e*FE)r"q
Entropie: 209.75 bits
● PS C:\Users\User> & C:/Users/User/AppData/Local/Programs/Python/Python314/python.exe c:/Users/User/Desktop/TD-/programmationweb/n
```

```
File Edit Selection View Go Search
PROBLEMS 7 OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\User> & C:/Users/User/AppData/Local/Programs/Python/Python314/python.exe c:/Users/User/Desktop/TD-/programmationweb/mod
Charset: ascii_lowercase
Clé (8 caractères): ogpfvnvo
Entropie: 37.60 bits
Clé (16 caractères): nmfwampyeulcvalg
Entropie: 75.21 bits
Clé (32 caractères): dgpzvibuvfbclknckhupepcjupxpnju
Entropie: 150.41 bits

Charset: ascii_letters
Clé (8 caractères): CkAtUREU
Entropie: 45.60 bits
Clé (16 caractères): ZvDfEwpBewmeNQej
Entropie: 91.21 bits
Clé (32 caractères): fXcYk0HPElFBFyFRbKQxHBPMEorwWIwV
Entropie: 182.41 bits

Charset: digits
Clé (8 caractères): 48754734
Entropie: 26.58 bits
Clé (16 caractères): 4327778878637314
Entropie: 53.15 bits
Clé (32 caractères): 28329244204352163874077074088770
Entropie: 106.30 bits

Charset: full
Clé (8 caractères): ,"|jC6?r
Entropie: 52.44 bits
Clé (16 caractères): )[_|\llw@1mrelM>
Entropie: 104.87 bits
Clé (32 caractères): E/S+]DiTc#Le&b|y+nnzNV1+mes?=xZ\
Entropie: 209.75 bits
○ PS C:\Users\User>
```

TD 2 : Chiffrement César

Objectif : Comprendre un chiffrement symétrique simple et la rotation des caractères.

Instructions :

1. Écrire une fonction `caesar_cipher(message, shift)` pour chiffrer un message.
2. Écrire la fonction inverse pour déchiffrer.
3. Tester sur des messages avec :
 - lettres minuscules
 - lettres majuscules
 - chiffres

```
File Edit Selection View Go ... Search

modelisation.py 2, U X

C: > Users > User > Desktop > TD- > programmationweb > modelisation.py > ...

1  # Fonction de chiffrement César
2  def caesar_cipher(message, shift):
3      result = ""
4      for char in message:
5          if char.islower(): # lettres minuscules
6              result += chr((ord(char) - ord('a') + shift) % 26 + ord('a'))
7          elif char.isupper(): # lettres majuscules
8              result += chr((ord(char) - ord('A') + shift) % 26 + ord('A'))
9          elif char.isdigit(): # chiffres
10             result += chr((ord(char) - ord('0') + shift) % 10 + ord('0'))
11         else:
12             result += char # caractères spéciaux non modifiés
13     return result
14
15  # Fonction de déchiffrement César
16  def caesar_decipher(message, shift):
17      return caesar_cipher(message, -shift)
18
19
20  # =====
21  # Tests
22  # =====
23
24  messages = [
25      "bonjour",      # minuscules
26      "HELLO",        # majuscules
27      "123456",       # chiffres
28      "SecuRite2026!" # mixte
29  ]
```

```
modelisation.py 2, U X
C: > Users > User > Desktop > TD- > programmationweb > modelisation.py > caesar_cipher

13     return result
14
15     # Fonction de déchiffrement César
16     def caesar_decipher(message, shift):
17         return caesar_cipher(message, -shift)
18
19
20     # =====
21     # Tests
22     # =====
23
24     messages = [
25         "bonjour",      # minuscules
26         "HELLO",        # majuscules
27         "123456",       # chiffres
28         "SecuRite2026!" # mixte
29     ]
30
31     shift = 3 # décalage choisi
32
33     for msg in messages:
34         encrypted = caesar_cipher(msg, shift)
35         decrypted = caesar_decipher(encrypted, shift)
36         print(f"Message original : {msg}")
37         print(f"Chiffré : {encrypted}")
38         print(f"Déchiffré : {decrypted}")
39         print("-" * 30)
40
```

```
PS C:\Users\User> & C:/Users/User/AppData/Local/Programs/Python/Python314/python.exe c:/Users/User/De
-----
Message original : SecuRite2026!
Chiffré : VhfxUlwh5359!
Déchiffré : SecuRite2026!
-----
PS C:\Users\User>
```


TD 3 Chiffrement XOR simple

Objectif : Comprendre le chiffrement par clé avec opérations bit à bit.

Instructions :

1. Générer une clé aléatoire de même longueur que le message en utilisant `string.printable`.
2. Écrire une fonction `xor_cipher(message, key)` pour chiffrer et déchiffrer.
3. Vérifier que le message déchiffré est identique à l'original.
4. Calculer l'entropie de la clé et discuter sa sécurité.

```
modelisation.py 3, U X
C: > Users > User > Desktop > TD- > programmationweb > modelisation.py > ...

1  import string
2  import secrets
3  import math
4
5  # Générer une clé aléatoire de même longueur que le message
6  def generate_key(length):
7      charset = string.printable # ensemble de caractères
8      return ''.join(secrets.choice(charset) for _ in range(length))
9
10 # Fonction de chiffrement/déchiffrement XOR
11 def xor_cipher(message, key):
12     return ''.join(chr(ord(m) ^ ord(k)) for m, k in zip(message, key))
13
14 # Calcul de l'entropie de la clé
15 def calculate_entropy(key, charset_size):
16     return len(key) * math.log2(charset_size)
17
18 # =====
19 # Exemple d'utilisation
20 # =====
21
22 message = "Bonjour123!"
23 key = generate_key(len(message))
24
25 print("Message original :", message)
26 print("Clé générée      :", key)
27
28 # Chiffrement
29 cipher_text = xor_cipher(message, key)
```

modelisation.py 3, U X

C: > Users > User > Desktop > TD- > programmationweb > modelisation.py > ...

```
13
14 # Calcul de l'entropie de la clé
15 def calculate_entropy(key, charset_size):
16     return len(key) * math.log2(charset_size)
17
18 # =====
19 # 🚀 Exemple d'utilisation
20 # =====
21
22 message = "Bonjour123!"
23 key = generate_key(len(message))
24
25 print("Message original :", message)
26 print("Clé générée      :", key)
27
28 # Chiffrement
29 cipher_text = xor_cipher(message, key)
30 print("Texte chiffré    :", cipher_text)
31
32 # Déchiffrement (XOR est symétrique)
33 decrypted = xor_cipher(cipher_text, key)
34 print("Texte déchiffré :", decrypted)
35
36 # Entropie de la clé
37 entropy = calculate_entropy(key, len(string.printable))
38 print(f"Entropie de la clé : {entropy:.2f} bits")
39
```



```
modelisation.py 3, U X
C: > Users > User > Desktop > TD- > programmationweb > modelisation.py > ...
13
PROBLEMS 11 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\User> & C:/Users/User/AppData/Local/Programs/Python/Python314/python.exe c:

Charset: ascii_lowercase
Clé (8 caractères): umxhxdkd
Entropie: 37.60 bits
Clé (16 caractères): euzebdoeploiknhv
Entropie: 75.21 bits
Clé (32 caractères): ezmdnwoxqaxzqfoyqflqcoofpyqpigwo
Entropie: 150.41 bits

Charset: ascii_letters
Clé (8 caractères): dVnjssBM
Entropie: 45.60 bits
Clé (16 caractères): EXJajCDwUzTSqMtY
Entropie: 91.21 bits
Clé (32 caractères): ROWoVyGVJEyfiuNXNouEgxjxnvelOBYm
Entropie: 182.41 bits

Charset: digits
Clé (8 caractères): 66017169
Entropie: 26.58 bits
Clé (16 caractères): 9704717868131012
Entropie: 53.15 bits
Clé (32 caractères): 79313551199323892692318001037530
Entropie: 106.30 bits

Charset: full
Clé (8 caractères): O#DU.+_"
Entropie: 52.44 bits
```

```
modelisation.py 3, U X
C: > Users > User > Desktop > TD- > programmationweb > modelisation.py > ...
13
PROBLEMS 11 OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\User> & C:/Users/User/AppData/Local/Programs/Python/Python314/python.exe c:/Users/User/Desktop/TD-/programmationweb/
Charset: full
Clé (8 caractères): O#DU.+_"
Entropie: 52.44 bits
Clé (16 caractères): +JyHz1)l,tMF\EJ,
Entropie: 104.87 bits
Clé (32 caractères): 1`;>=D`7z'2d4E2qdViA}FeS,WpWL+o{
Entropie: 209.75 bits
PS C:\Users\User> & C:/Users/User/AppData/Local/Programs/Python/Python314/python.exe c:/Users/User/Desktop/TD-/programmationweb/

Charset: ascii_lowercase
Clé (8 caractères): oavawjdi
Entropie: 37.60 bits
Clé (16 caractères): nufzutjqdgkqejcb
Entropie: 75.21 bits
Clé (32 caractères): jslpqxjurxjvxdavdwgllmtlvmdxymwg
Entropie: 150.41 bits

Charset: ascii_letters
Clé (8 caractères): qkDzyOak
Entropie: 45.60 bits
Clé (16 caractères): zfVKakyBboQNCqpq
Entropie: 91.21 bits
Clé (32 caractères): UkjLHlLvNyChSmUctXDUszaogUzTeDL
Entropie: 182.41 bits

Charset: digits
Clé (8 caractères): 45819114
Entropie: 26.58 bits
```

```
modelisation.py 3, U X
C: > Users > User > Desktop > TD- > programmationweb > modelisation.py > ...
13
PROBLEMS 11 OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\User> & C:/Users/User/AppData/Local/Programs/Python/Python314/python.exe c:/Users/User/Desktop/TD-/programmationweb/mo
Entropie: 104.87 bits
Clé (32 caractères): }*vMn2Hu{grg(@`cLp5M?Id'e*FE)r"ng
Entropie: 209.75 bits
PS C:\Users\User> & C:/Users/User/AppData/Local/Programs/Python/Python314/python.exe c:/Users/User/Desktop/TD-/programmationweb/mo

Charset: ascii_lowercase
Clé (8 caractères): nshqwzzh
Entropie: 37.60 bits
Clé (16 caractères): oxeqhffdbstamfch
Entropie: 75.21 bits
Clé (32 caractères): vdtwjoqykfpfnjtpkavcpowuvxjftqks
Entropie: 150.41 bits

Charset: ascii_letters
Clé (8 caractères): NNsVf1Dm
Entropie: 45.60 bits
Clé (16 caractères): xpiGOOqfTCujkvDL
Entropie: 91.21 bits
Clé (32 caractères): lubozJiLocbOkBWXdEnbpxrmYFCLAGKZ
Entropie: 182.41 bits

Charset: digits
Clé (8 caractères): 21788962
Entropie: 26.58 bits
Clé (16 caractères): 7467449182010858
Entropie: 53.15 bits
Clé (32 caractères): 74319862668556932057547859094076
Entropie: 106.30 bits
```

```
modelisation.py 3, U X
C: > Users > User > Desktop > TD- > programmationweb > modelisation.py > ...
13
PROBLEMS 11 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\User> & C:/Users/User/AppData/Local/Programs/Python/Python314/python.exe c:/Users
Entropie: 53.15 bits
Clé (32 caractères): 74319862668556932057547859094076
Entropie: 106.30 bits

Charset: full
Clé (8 caractères): gbN/7.10
Entropie: 52.44 bits
Clé (16 caractères): uUky~k=zVUr1cB!~
Entropie: 104.87 bits
Clé (32 caractères): qH\Z9"8/Mw#TuCwf\$0Ut8*1T)r.U+3#
Entropie: 209.75 bits
Message original : bonjour
Chiffré : erqmrXu
Déchiffré : bonjour
-----
Message original : HELLO
Chiffré : KHOOR
Déchiffré : HELLO
-----
Message original : 123456
Chiffré : 456789
Déchiffré : 123456
-----
Message original : SecuRite2026!
Chiffré : VhfxUlwh5359!
Déchiffré : SecuRite2026!
-----
PS C:\Users\User> & C:/Users/User/AppData/Local/Programs/Python/Python314/python.exe c:/Users
```

```
modelisation.py 3, U X
C: > Users > User > Desktop > TD- > programmationweb > modelisation.py > ...
13
PROBLEMS 11 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\User> & C:/Users/User/AppData/Local/Programs/Python/Python314/python.exe c:/User
Message original : HELLO
Chiffré : KHOOR
Déchiffré : HELLO
-----
Message original : 123456
Chiffré : 456789
Déchiffré : 123456
-----
Message original : SecuRite2026!
Chiffré : VhfxUlwh5359!
Déchiffré : SecuRite2026!
-----
● PS C:\Users\User> & C:/Users/User/AppData/Local/Programs/Python/Python314/python.exe c:/User
Message original : Bonjour123!
Clé générée :
&f.P9jOQUY
Texte chiffré : HD?L~cfx
Texte déchiffré : Bonjour123!
Entropie de la clé : 73.08 bits
● PS C:\Users\User> & C:/Users/User/AppData/Local/Programs/Python/Python314/python.exe c:/User
Message original : Bonjour123!
Clé générée : 0n2mT>A?bwz
Texte chiffré : r\;K3PD[
Texte déchiffré : Bonjour123!
Entropie de la clé : 73.08 bits
○ PS C:\Users\User> 
```

TD 4 : Génération et visualisation de l'entropie

Objectif : Relier entropie, longueur de clé et alphabet.

Instructions :

1. Générer des clés de longueurs différentes (4, 8, 16, 32).
2. Pour chaque clé, calculer l'entropie.
3. Tracer un graphique entropie vs longueur de clé avec matplotlib.
4. Discuter la relation entre longueur de clé, alphabet et sécurité.

```
modelisation.py 4, U X
C: > Users > User > Desktop > TD- > programmationweb > modelisation.py > ...

1  import string
2  import secrets
3  import math
4  import matplotlib.pyplot as plt
5
6  # Fonction pour générer une clé aléatoire
7  def generate_key(length, charset):
8      return ''.join(secrets.choice(charset) for _ in range(length))
9
10 # Fonction pour calculer l'entropie
11 def calculate_entropy(key, charset_size):
12     return len(key) * math.log2(charset_size)
13
14 # Alphabet utilisé (lettres, chiffres, ponctuation)
15 charset = string.ascii_letters + string.digits + string.punctuation
16 charset_size = len(charset)
17
18 # Longueurs de clés à tester
19 lengths = [4, 8, 16, 32]
20 entropies = []
21
22 # Génération des clés et calcul des entropies
23 for length in lengths:
24     key = generate_key(length, charset)
25     entropy = calculate_entropy(key, charset_size)
```


modelisation.py 4, U X

C: > Users > User > Desktop > TD- > programmationweb > modelisation.py > ...

```
14 # Alphabet utilisé (lettres, chiffres, ponctuation)
15 charset = string.ascii_letters + string.digits + string.punctuation
16 charset_size = len(charset)
17
18 # Longueurs de clés à tester
19 lengths = [4, 8, 16, 32]
20 entropies = []
21
22 # Génération des clés et calcul des entropies
23 for length in lengths:
24     key = generate_key(length, charset)
25     entropy = calculate_entropy(key, charset_size)
26     entropies.append(entropy)
27     print(f"Clé ({length} caractères): {key}")
28     print(f"Entropie: {entropy:.2f} bits\n")
29
30 # Tracer le graphique Entropie vs Longueur de clé
31 plt.plot(lengths, entropies, marker='o', linestyle='-', color='b')
32 plt.title("Entropie vs Longueur de clé")
33 plt.xlabel("Longueur de la clé (caractères)")
34 plt.ylabel("Entropie (bits)")
35 plt.grid(True)
36 plt.show()
37
```


